Scenario-Based SQL Questions – Set 4

Practical SQL scenarios • Questions and logic explanations

Scenario 1: Identify Loyal Customers

Question:
Find customers who have made purchases in each of the last 12 months within the same calendar year.

Logic Explanation:

Join the customers table with the sales table using customer_id.

Use YEAR() and MONTH() on sale_date to get the year and month of each purchase.

Apply GROUP BY on customer and YEAR(sale_date).

Use COUNT(DISTINCT MONTH(sale_date)) to count in how many different months the customer purchased.

Use HAVING COUNT(DISTINCT MONTH(sale_date)) = 12 to keep customers who purchased in all 12 months of that year.

Concepts Used:

Joins (INNER JOIN)

Date Functions (YEAR(), MONTH())

Aggregation (COUNT(DISTINCT))

Filtering Groups (HAVING)

Scenario 2: Detect Revenue Anomalies

Question:
Identify products whose sales amount increased by more than 50% in a month compared to the previous month, using a self-join.

Logic Explanation:

First calculate the total sales of each product for each month using SUM(amount) with GROUP BY product_id and DATE_FORMAT(sale_date, '%Y-%m').

Perform a self-join on this monthly result using product_id.

Match each month with its previous month using sale_date - INTERVAL 1 MONTH.

Calculate the growth percentage: (current_month - previous_month) / previous_month * 100.

Filter products where the current month sales are more than 1.5 times the previous month sales.

Concepts Used:

Self Join

Aggregation (SUM())

Date Functions (DATE_FORMAT(), INTERVAL)

Filtering Groups (HAVING)

Scenario 3: Employee Promotion Tracking

Question:
Find employees who have been promoted more than once within the same calendar year.

Logic Explanation:

Join the promotions table with the employees table using employee_id.

Use YEAR(promotion_date) to get the year of each promotion.

Apply GROUP BY on employee name and YEAR(promotion_date).

Use HAVING COUNT(*) > 1 to find employees with more than one promotion in the same year.

Concepts Used:

Joins (INNER JOIN)

Date Functions (YEAR())

Aggregation (COUNT())

Filtering Groups (HAVING)

Scenario 4: Find Inactive Products

Question:
List products that have not had any sales in the last 6 months.

Logic Explanation:

Use a subquery to find the product_id of all products sold in the last 6 months using WHERE sale_date >= CURDATE() - INTERVAL 6 MONTH.

Select products from the products table whose product_id is NOT IN the subquery result.

This also includes products that have never been sold.

Display product_id and product_name.

Concepts Used:

Subqueries

Filtering (NOT IN)

Date Functions (CURDATE(), INTERVAL)

Scenario 5: Detect Customer Churn

Question:
Identify customers who haven't made any purchases in the last 90 days, including customers who have never made a purchase.

Logic Explanation:

Perform a LEFT JOIN between the customers table and the sales table using customer_id, so customers without any sales are also kept.

Apply GROUP BY customer_id and use MAX(sale_date) to find each customer's last purchase date.

Use HAVING MAX(sale_date) IS NULL to include customers who never purchased.

Add OR MAX(sale_date) < CURDATE() - INTERVAL 90 DAY to include customers whose last purchase was more than 90 days ago.

Concepts Used:

Joins (LEFT JOIN)

Aggregation (MAX())

Filtering with NULL Values

Date Functions (CURDATE(), INTERVAL)